Link Check

Offline website and model guide

This package contains an HTML website with the trained Decision Tree inside it. You can check URLs without installing Python or starting a server. The original Link Check layout is preserved.

Open the website

1. Extract the ZIP to a normal folder.

2. Double-click **Link_Check.html**. Open it in a current Chrome, Edge, Firefox, or Safari browser.

3. Paste an address and click **Check link**.

4. Read the score, the model label, and the URL observations.

5. Open **History** to see results from this browser. Use **Export my history as JSON** to keep a copy.

No terminal, conda, Flask, account, internet connection, or installation is needed. JavaScript must be enabled in the browser. Open the extracted HTML, rather than its preview inside the ZIP.

Try these examples

Input	Expected result from this model
https://www.google.com	70.3 / 100 - low risk
roblox.com	70.3 / 100 - low risk
http://192.0.2.1/login	15.0 / 100 - high risk
bukan link	Invalid input

History and privacy

The file does not visit or send the entered URL anywhere. History is kept in browser local storage when available. Some browsers restrict storage for local files or private windows. If saving is unavailable, checking still works for the current session. Export results to keep them. Moving the HTML may start a separate history.

Report this result marks an entry for local review. It does not send a report to another organisation. Model outputs are not verified training labels.

How the model works

The browser runs the same saved tree rules as the Python classifier. The HTML contains the feature names, split thresholds, branches, and phishing proportions at its leaves. It does not train a new model or invent a score.

Data and features

Training and evaluation used 30,000 real labelled URLs: 20,000 from the assignment Kaggle source and 10,000 from UCI PhiUSIIL. UCI labels were reversed to match benign = 0 and phishing = 1. Only raw URLs and labels were imported; existing source features were excluded. Equivalent formats and conflicting duplicate labels were removed before sampling.

The scheme and leading www are excluded from scoring to reduce formatting bias. Eleven numerical features describe address length, hostname length, dots, URL hyphens, at signs, digit ratio, path depth, query length, an IP host, a broad common-TLD category, and hostname hyphens. These are signals, not proof of legitimacy.

Score and model limits

Trust score = 100 x (1 - model phishing probability). Below 40 is high risk, 40 to below 70 is caution, and 70 or more is low risk. These display bands are not validated security thresholds. Different URLs can get the same score because they reach the same tree leaf.

The revised development test accuracy is **70.5%**; phishing recall is **73.0%**. The exact-hostname holdout accuracy is **68.4%**. Revisions followed feedback, so these are development comparisons, not untouched external validation. A high score does not guarantee safety.

No webpage content, SSL certificate, domain age, brand impersonation, or current threat feed is checked. URL observations are separate simple checks, not a causal explanation of the prediction. K-Means clustering was used in the training project for analysis and is not part of the browser verdict.

What is in this package

The package includes **Link_Check.html**, this guide, and a **Python_Project** folder with Python source, datasets, saved models, and evaluation outputs. The HTML is a ready-to-run demo; Python is included for analysis and retraining. A final assignment report and meeting records are not included.

Sources and development note

sid321axn (n.d.), Malicious URLs dataset, Kaggle.
Kaggle dataset source

Prasad, A. and Chandra, S. (2024), PhiUSIIL Phishing URL (Website), UCI Machine Learning Repository. CC BY 4.0; raw URLs and labels transformed and sampled.
UCI dataset source

scikit-learn developers (n.d.), Decision Trees.
Model documentation

Code preparation used AI assistance. Team members should review the implementation, understand the results, and acknowledge assistance according to the unit policy. Sources viewed 10 October 2026.

Python analysis and training

Open a terminal inside the extracted Python_Project folder. Install Miniconda or Anaconda if conda is not available.

Set up the environment

```
conda create -n phishing-project python=3.12 -y
conda activate phishing-project
pip install -r requirements.txt
```

Run training and prediction

```
python train.py
python predict.py "https://www.google.com"
```

Training uses the included prepared URLs and works offline. It writes new models and evaluation files in models and results. Read metrics.json, source_metrics.json, errors.csv, cluster_means.csv, and cluster_examples.csv for your analysis. The plots and tree_rules.txt help explain the trained model.

Rebuild the dataset

```
python prepare_data.py
python train.py
```

This step downloads Kaggle and UCI source archives and needs internet access. features.py contains the shared normalisation and numerical feature functions. urls.csv contains raw selected URL text and labels; provenance.csv gives the source for each row. features.csv is the numerical feature table. source.json records transformations, counts, and archive hashes.

Optional Flask website

```
python app.py
```

Open http://127.0.0.1:5050 while the terminal is running. This version reads the Python model and stores local history in SQLite. Use Ctrl+C to stop it. The standalone HTML uses browser storage instead. Original private history and session secrets are excluded from the ZIP.

Updating the standalone HTML model

After Python retraining, the original HTML still contains its earlier exported tree. Run `python export_html.py` to update the adjacent Link_Check.html file. Keep it beside this folder. The exporter embeds the new branches, thresholds, feature list, and leaf probabilities. Feature changes also require matching changes to the browser feature function; the exporter cannot translate arbitrary Python code.